# 2.1 前導片-失傳的設計模式.mp4

## 📐 軟體設計模式的起源與本質

> 「每個模式都是由三個部分組成的規則，描述在特定情境（Context）下，遇到某個問題（Problem）時，如何透過某個解決方案（Solution）來解決該問題。」

### 1. 設計模式的起源：Christopher Alexander 的建築模式思想

- **Christopher Alexander** 是建築學者，他在《永恆的建築之道》（The Timeless Way of Building）中首次提出「設計模式（Pattern）」的概念。
- 設計的核心是針對問題提出有效的解決方案。
- 這些解決方案受到「人事時地物」即**情境（Context）**的限制，不同情境下，相同問題需要不同的解決方案。
- **範例說明：**
  - 一般人想要溫暖和陪伴，可能會用火爐和舒適客廳來實現。
  - 老奶奶需要的溫暖是有人陪伴，火爐反而凸顯孤獨，不適用。
- Alexander 認為，這些反覆出現的問題和解決方案形成了「模式」，能被整理並流傳使用。

### 2. 軟體設計模式的發展：四人幫（Gang of Four）

- 軟體領域的設計模式靈感來自 Alexander 的思想。
- 1994 年，四人幫聯合出版《Design Patterns: Elements of Reusable Object-Oriented Software》，系統化整理了 23 個經典設計模式（如策略模式、單體模式、狀態模式等）。
- 這本書被視為軟體設計模式的「聖經」，對許多工程師影響深遠。

### 3. 四人幫設計模式的結構與內容

| 項目 | 內容說明 |
| --- | --- |
| 名稱 | 設計模式的名稱（如策略模式 Strategy Pattern） |
| 動機（Intent） | 為何要使用此設計模式，解決什麼問題 |
| 別名（As Known As） | 該模式的其他稱呼 |
| 發明典故（Motivation） | 模式誕生的背景故事 |
| 適用情境（Applicability） | 何時該使用此模式 |

| | |
|---|---|
| 結構（Structure） | 類別圖結構，描述參與者及其角色 |
| 參與者（Participants） | 模式中各角色的職責 |
| 互動（Collaborations） | 參與者間的合作方式 |
| 優缺點（Consequences） | 採用此模式的利與弊 |
| 實作細節（Implementation） | 具體實作建議 |
| 已知案例（Known Uses） | 真實世界中此模式的使用情況 |

Thetawave.ai

- 雖然內容完整，但更多是百科式的記錄，缺乏引導思考的步驟。

## ⚠️ 設計模式造成的困擾與誤用

### 4. 學習設計模式後的「選擇障礙」

- 許多工程師學完設計模式後，反而覺得困惑且難以選擇合適模式，產生「選擇障礙」。
- **比喻：逛夜市選擇困難症**
  - 面對 23 個攤販（模式），每個都想嘗試但又怕踩雷。
  - 選錯模式就像買到雷店，程式變複雜且效果不佳。
- 盲目套用設計模式容易導致程式難以維護，且無法清楚說明為何使用該模式。

### 5. 設計模式誤用的根本原因

- 現代工程師多只依據模式的「動機（Intent）」來選擇模式。
- 常見誤解是「只要有這種動機，就直接套用某個模式」：

| 動機描述 | 常用模式 |
|---|---|
| 想消除 if-else | 策略模式（Strategy Pattern） |
| 類別有多種狀態 | 狀態模式（State Pattern） |
| 需要建立不同物件 | 工廠方法（Factory Method） |
| 想有唯一實例 | 單體模式（Singleton Pattern） |
| 封裝一系列指令 | 指令模式（Command Pattern） |
| 想要 callback 機制 | 觀察者模式（Observer Pattern） |
| 動態添加行為 | 裝飾者模式（Decorator Pattern） |

- 這種選擇方式將設計變成憑個人「覺得」或「動機」行事，忽略了真正的需求與情境，類似「酒駕」般危險。

### 6. 設計不可憑主觀動機行事

- **設計應基於客觀需求，而非主觀感覺。**
- 比喻：開車不能隨意左轉，需要看路況並遵守規則。
- 在軟體設計中，必須根據**需求分析（需求）**和情境（Context）**做決策。
- 在程式碼審查（code review）時，不該以「我覺得」為理由，而要以「我根據需求」來說明設計選擇。

- 違反這點即被稱為「酒駕」，需被糾正。

Thetawave.ai

## 🎯 設計模式應用的正確思維與展望

### 7. 軟體設計的愉悅與「剛剛好」的平衡

- 設計不必追求完美或過度套模式，應該是「剛剛好」——**既精準又優雅**。
- 當需求明確且有必要時，才套用設計模式；不必要時，則不強求。

### 8. 失傳的關鍵：Forces（約束）

- Christopher Alexander 強調設計的本質是**辨識約束（Forces）之間的衝突，並透過解決方案釋放這些衝突**。
- 四人幫書中未充分引入「Forces」思想，導致軟體設計模式應用上缺少了這層重要理解。
- 後續課程將深入探討「Forces」概念，幫助理解如何根據約束做出設計決策。

---

設計模式的核心在於理解問題與情境的限制（Context）及多種約束（Forces），並尋找合適的解決方案（Solution）來平衡這些衝突，從而達成剛剛好且優雅的軟體設計。盲目憑主觀動機選擇模式會導致設計混亂，唯有基於客觀需求與約束的分析，才能真正發揮設計模式的效用。

## 2.2 架構師該學的 C.A. 模式六大要素及模式思維.mp4

## 🎯 Christopher Alexander 的模式六大要素與模式語言

### 1. 模式（Pattern）的基礎概念

> **模式（Pattern）定義：**
> 模式是一種解決特定問題的通用方案，由三個部分組成的「三段式規則（Three Part Rule）」：
>
> **Context（情境）**：問題發生的背景或前提條件
>
> **Problem（難題）**：在該情境下遇到的具體困難
>
> **Solution（解法）**：針對問題的具體解決方案

生活中的案例說明

以「小明搭訕心儀女生」為例：

- **Context**：小明想要取得女生的聯絡方式，且女生習慣使用 Line、喜歡撿便宜、喜歡收到禮物。
- **Problem**：小明害羞、口才不好、不上相，無法直接搭訕。

- **Solution**：用「送快過期星巴克 Line Gift」的方式，藉由送禮交換聯絡方式。Thetawave.ai

這個模式的成功率約87%，但要注意解法只能在符合 Context 的情況下有效。

---

## 2. 模式的六大要素（六大構件）

| 要素名稱 | 說明 | 小明案例中的具體內容 |
|---|---|---|
| **Name（名字）** | 模式的名稱，方便識別和溝通 | Line Gift Pattern（送禮換聯絡方式模式） |
| **Context（情境）** | 問題產生的前提、環境、條件 | 對方習慣用 Line、喜歡撿便宜、喜歡禮物 |
| **Forces（約束）** | 制約問題的多項衝突條件或因素 | 小明害羞、口才差、不上相、怕錯過機會等 |
| **Problem（問題）** | 由衝突的 Forces 形成，必須解決的核心難題 | 如何在這些限制下取得聯絡方式 |
| **Solution（解法）** | 解決上述問題的具體方法與步驟 | 用 Line Gift 送禮，藉此交換聯絡方式 |
| **Resulting Context（結果情境）** | 套用解法後產生的新情境與後續問題 | 成功拿到 Line，但仍需面對後續聊天、破冰等新問題 |

這六大要素能系統化描述每一個模式，使設計決策清晰且標準化。

---

## 3. Problem 的嚴謹定義與設計決策的核心

> **定義：Problem = 一組衝突的 Forces（約束）**

- **Forces** 是客觀存在、限制設計的因素。
- 只有當多個 Forces 彼此產生衝突時，才構成 Problem。
- 若只有單一 Force，或多個 Forces 但不衝突，則不構成 Problem，無需花資源解決。

例子解釋

| 情境說明 | Forces 數量 | 是否構成 Problem | 理由 |
|---|---|---|---|
| 帥哥想要搭訕，只需一個顧慮 | 1 | 否 | 沒有衝突，問題簡單，不是 Problem |
| 小明害羞、不上相等多重限制 | 4 | 是 | 多個互相矛盾的限制，形成難題 |

設計決策的準確度很大程度取決於對 Forces 的察覺與分析。

---

## 4. Forces、Problem 與 Solution 的關係流程

Thetawave.ai

1. **察覺 Context 中的多個 Forces**
2. **衝突的 Forces 形成 Problem**
3. **針對 Problem 提出 Solution（解法）**
4. **套用 Solution 後產生新的 Resulting Context**

這個流程可用咒語記憶：
「從 Context 中察覺 Forces，衝突的 Forces 構成 Problem，提出 Solution 解決 Problem，得到 Resulting Context。」

---

### 5. 模式語言（Pattern Language）的概念

- **定義：** 一組相關的模式集合。這些模式之間存在「先後關係」、「互補關係」、「組合關係」等各種關聯。
- **作用：** 幫助設計者在不同情境中自由組合多個模式，形成一整套解決方案或設計套路。
- **比喻：** 就像語言一樣，由字詞組合成句子，模式語言則由多個模式組合成完整的設計思想。

在小明搭訕的例子中，Line Gift Pattern 只是第一步，後續還有破冰模式和閃光點模式，這些模式合起來構成「搭訕的模式語言」。

---

### 6. 多模式連環套用與設計思維

- 一個模式解決一個 Problem，套用完後進入新的 Resulting Context，可能產生新的 Problem，需要用其他模式繼續解決。
- 多個模式串聯使用，形成連鎖反應，提升整體解決方案的成功率。
- 這種思維能幫助設計者在複雜系統中，逐步化解層層難題。

---

### 7. 軟體設計中的模式思考與 OADP 框架

- **OADP 定義：** Pattern Adaptive Object Oriented Analysis and Design（適應模式的物件導向分析與設計）
- **步驟：**
  1. 需求驅動，排除主觀判斷
  2. 用 OA（物件分析）區分結構與行為
  3. 用設計模型（如 UML 類別圖、序列圖）去除雜訊，清晰呈現架構

Thetawave.ai

4. 察覺並盤整客觀的 Forces（約束）
5. 定義 Problem（多個衝突的 Forces）
6. 套用設計模式（Solution/Form）解決問題
7. 迭代設計直到無尚未解決的 Problem

- **設計模式中的 Solution 也稱為 Form（形式）**：因為在建築領域，Solution 通常是具體的形狀或結構；軟體設計模式中，類別圖和序列圖即是這種有形的形式。

---

### 8. Forces 在軟體設計中的種類與應用

| Forces 分類 | 說明 | 影響範圍 |
| --- | --- | --- |
| 需求變動點 | 需求可能改變的不同方式，分為六種更嚴格定義 | 需求與架構的適應性與彈性 |
| 其他軟體設計約束 | 例如效能、擴充性、安全性、可測試性等 | 軟體品質與架構設計決策 |

- 這些 Forces 都是客觀且可察覺的，能快速辨識以判斷是否需要套用特定設計模式。
- 透過 Forces 的組合查表，即可快速找到相應的設計模式，提高設計效率。

---

### 9. 小結：模式六大要素與設計決策的核心

- **模式六大要素**提供結構化方法，幫助設計者明確定義問題與解決方案。
- **Problem 必須是衝突的 Forces 組合**，只有這種狀況才值得投入資源解決。
- **模式語言**則由多個相關模式組成，促進複雜系統的分層解決。
- **軟體設計中，OADP 框架結合模式思想，實現需求驅動、客觀分析與高效決策。**
- **Forces 的察覺與盤整是設計決策的關鍵，設計模式的正確套用依賴於對 Forces 的深刻理解。**

這套理論體系將設計視為多因素衝突的管理與解決過程，將抽象的設計問題具象化，促使設計決策更精準且有效。

## 🎯 軟體設計中的 Forces 與設計模式應用

> 「設計模式（Design Pattern）是解決軟體設計中反覆出現問題的最佳方案，透過理解並應用這些模式，可以有效化解設計過程中遭遇的各種力量（forces）。」

---

## 1. 設計中面臨的三大技術性 Forces

| Forces | 說明 | 影響 |
| --- | --- | --- |
| **變動性（Volatility）** | 技術演進導致系統元件（如資料庫驅動程式）可能被替換或更新。 | 系統因外部依賴變動而可能崩潰或功能失效。 |
| **擴充性（Extensibility）** | 當替換系統元件時，若需要修改核心程式碼，會增加維護與擴展成本。 | 修改核心程式碼的成本高，降低系統靈活度。 |
| **相容性（Compatibility）** | 外部依賴（如第三方資料庫驅動）無法被修改，限制系統的整合方式。 | 系統必須適應且不破壞外部套件的接口，增加設計難度。 |

Thetawave.ai

### 1.1 解耦遊戲類別與資料庫驅動的問題

- **問題核心**：如何在不改動遊戲（game）核心程式碼的情況下，更換資料庫驅動。
- **解決方案**：採用 **Adapter Pattern（適配器模式）** 來包裝不同的資料庫驅動，使遊戲與資料庫驅動解耦。

### 1.2 Adapter Pattern 的重構三步驟

1. **Encapsulate What Varies（封裝變動部分）**
   將可變部分（不同資料庫驅動）封裝成獨立類別。
2. **Abstract Common Behavior（抽象共同行為）**
   定義一組共同的接口，讓不同的驅動能被統一操作。
3. **Delegation（委派）**
   讓遊戲類別委派資料操作給適配器，達成解耦。

---

## 2. 領域需求帶來的複雜性 Forces 與行為組合

| Forces | 說明 | 影響 |
| --- | --- | --- |
| **變動性（Volatility）** | 怪物行為組合多樣，且常常變動。 | 直接繼承法導致類別組合爆炸，難以維護。 |
| **維護性（Maintainability）** | 類別過多導致後續維護與擴展困難。 | 程式碼複雜、重複且不易擴展。 |

### 2.1 解決怪物行為組合的問題

- **問題核心**：如何讓怪物擁有多種行為組合，且避免類別組合爆炸。
- **解決方案**：使用 **Decorator Pattern（裝飾器模式）**，動態組合怪物行為。

### 2.2 Decorator Pattern 的特點

- 以「包裝」的方式動態添加行為，避免繼承層級膨脹。
- 允許自由組合走路、跑步、跳躍、追擊、打鬥等行為。
- 透過行為的動態組合提升系統彈性與維護性。

---

Thetawave.ai

## 3. 面對網路封包處理的設計挑戰

| Forces | 說明 | 影響 |
| --- | --- | --- |
| **變動性（Volatility）** | 網路封包種類及內容多變，封包處理需頻繁調整。 | 直接在遊戲類別中寫大量 if-else 條件判斷，造成程式碼混亂。 |
| **擴充性（Extensibility）** | 新增或修改封包處理規則時，需修改遊戲核心程式碼，擴充成本高。 | 核心類別變動頻繁，影響穩定性與可維護性。 |

### 3.1 解決封包處理擴充性的問題

- **問題核心**：如何在不修改遊戲核心類別的前提下，擴充封包處理規則。
- **解決方案**：採用 **Chain of Responsibility Pattern（責任鏈模式）**，將封包處理責任串聯。

### 3.2 Chain of Responsibility Pattern 的運作方式

- 多個處理器按序串聯，封包依序通過。
- 每個處理器負責一部分封包處理，符合條件即處理，否則交給下一個處理器。
- 大幅減少 if-else 判斷，提升程式碼清晰度與擴展彈性。

---

## 4. 設計模式、Forces 與思路的三大維度

| 維度 | 說明 |
| --- | --- |
| **熟悉模式量** | 熟悉越多設計模式，能選用合適模式解決不同問題。 |
| **理解各種 Forces** | 明白軟體設計中常見的變動、擴充、維護等力量，能準確判斷設計痛點。 |
| **思路行雲流水** | 具備良好設計思路，能拆解問題、分析 forces，並依據模式語言順暢地推進設計。 |

### 4.1 AI 與人類設計能力的差異

- AI 可熟悉大量設計模式與其應用情境，但缺乏拆解問題、判斷關鍵 forces 的思路。
- 人類設計高手能清晰拆解需求，推理並決策模式的適用，成為 AI 的駕駛者。
- 因此，培養設計思路與理解 forces 是成為卓越軟體設計師的關鍵。

---

## 5. 重要概念彙整

| 概念 | 定義 | 典型應用範例 |
| --- | --- | --- |
| **Forces（設計力量）** | 影響系統設計的各種內外部變動因素，如變動性、擴充性、相容性等。 | 技術更新需替換資料庫驅動、怪物行為多樣演進、封包類型頻繁變化等。 |

| | | |
|---|---|---|
| **Adapter Pattern（適配器模式）** | 將不同接口的類別透過包裝轉換成統一接口，解耦系統與外部依賴。 | 將 MySQL 驅動封裝成統一介面，避免遊戲核心直接依賴驅動細節。 |
| **Decorator Pattern（裝飾器模式）** | 動態組合物件行為，避免繼承帶來的類別爆炸，提升系統彈性與維護性。 | 怪物行為自由組合走路、跑步、跳躍等多種行為。 |
| **Chain of Responsibility Pattern（責任鏈模式）** | 多個處理器串聯請求，按序處理，減少條件判斷，方便擴充與維護。 | 網路封包依序通過多個處理器，對不同封包類型做對應處理。 |
| **Pattern Language（模式語言）** | 一組彼此關聯的設計模式集合，形成一套系統化的設計方法論。 | 大型多人線上遊戲中，各設計模式組合使用形成高品質軟體架構。 |

---

## 6. 比喻說明：設計模式與 Forces 的關係

將設計模式比喻為「解決問題的工具箱」，而 forces 則是「不同的環境挑戰」。就像在建築中遇到風、雨、地震等自然力量，設計師需選擇合適材料與結構來應對；軟體設計中，設計師需選擇合適的設計模式來化解系統中各種 forces，確保系統穩定且易於擴展。

---

設計模式的應用緊密結合系統面臨的各種 forces，透過重構步驟與模式語言，能在不斷變動的技術與需求中保持系統高品質與高彈性。熟悉模式本身不夠，必須具備拆解 forces 與推理決策的能力，才能持續推動設計思路，成為軟體設計的高手。

## 2.5 景點 - 策略模式_基礎水平行為擴充.mp4

## 🎯 策略模式 (Strategic Pattern) 基本介紹與應用場景

> 策略模式是一種行為型設計模式 (Behavioral Pattern)，用於封裝一組可自由切換的演算法或行為，讓客戶端（Client）能夠輕鬆擴充與替換行為，並且提升程式的可維護性和擴充性。

---

## 需求描述與初版設計

### 遊戲需求重點

| 項目 | 說明 |
|---|---|
| 遊戲類型 | 雙人英雄對戰 |
| 英雄屬性 | 名稱 (Name)、血量 (HP)，初始 HP 為 500，且不低於 0 |
| 攻擊方式 | 不同英雄有不同攻擊方式，未來持續新增攻擊方式 |
| 遊戲流程 | 兩位英雄輪流攻擊，任一方 HP ≤ 0 即判定死亡，遊戲結束 |
| 初版攻擊 | 水球攻擊(Waterball)：1 次，傷害 = 自身血量 * 0.5<br>火球攻擊(Fireball)：3 次，傷害皆為 50 |

| | |
|---|---|
| | 地球攻擊(Earth)：10 次，傷害皆為 20 |

Thetawave.ai

### 初版類別圖與程式碼問題點

- **Hero** 類別擁有 Attack 方法，透過 Switch Case 根據 Attack Type 字串執行不同攻擊行為
- **Game** 類別負責遊戲流程控制，兩英雄輪流攻擊
- 程式碼中攻擊行為寫死在 Hero 類別中，導致未來新增攻擊方式必須修改 Hero 類別

---

## 面臨的三大 Forces（設計動力）

| Force 名稱 | 說明 |
|---|---|
| 行為變動性 (Behavioral Variation) | 攻擊行為會隨 Attack Type 不斷變化且持續新增 |
| 維護性 (Maintainability) | 想讓 Hero 類別保持簡潔，避免頻繁修改以利閱讀與維護 |
| 擴充性 (Extensibility) | 希望能簡單擴充新的攻擊行為，且不影響 Hero 類別 |

這三個 Forces 彼此衝突，造成程式難以維護與擴展。

---

## 問題定義（Problem）

> 如何擴充 Hero 不同 Attack Type 的行為，且不影響 Hero 類別內部程式碼？

---

## 策略模式的解決方案 (Form / Solution)

策略模式透過三個重構步驟實現 **依賴反轉** (Dependency Inversion) 來解決問題：

### 依賴反轉三步驟

| 步驟 | 說明 |
|---|---|
| 1. 封裝變動之處 (Encapsulate What Varies) | 將不同攻擊行為封裝成獨立類別，每個類別代表一種攻擊行為變種（Behavioral Variant），只暴露 Attack 操作 |
| 2. 抽象共同行為 (Abstract Common Behaviors) | 觀察攻擊變種共同行為，定義一個抽象介面 (Interface)，所有攻擊類別都實作此介面 |
| 3. 委派 (Delegation) | Hero 類別不直接實作攻擊行為，而是委派給該介面實作物件，形成 Hero 依賴 Attack Type 介面 |

### 為何稱為依賴反轉？

- 初版：Hero 類別依賴具體攻擊行為（依賴方向為 Hero → Attack 行為）

- 重構後：攻擊行為依賴抽象介面，Hero 依賴介面，形成抽象依賴，且攻擊行為實作該介面（依賴方向反轉為 Attack行為 → Hero 抽象介面）

Powered by Wave.ai

---

## 依賴注入 (Dependency Injection)

- 將 Attack Type 介面實例由 Hero 類別的建構子或 Setter 傳入
- Hero 不自行決定攻擊行為實例，透過外部注入 (Inject) 方式取得依賴物件
- 簡化 Hero 類別的依賴管理，促進擴充與替換行為的彈性

---

## 第二版類別圖與程式碼架構

| 角色 | 說明 |
| --- | --- |
| Context (Hero) | 持有 Attack Type 介面引用，並委派攻擊行為給該介面實作物件 |
| Strategy 介面 | Attack Type 抽象介面，定義攻擊動作 Attack(Hero attacker, Hero target) |
| Concrete Strategies | 各種具體攻擊類別 (Waterball, Fireball, Earth, Powerball 等) 實作 Strategy 介面 |

---

## 使用策略模式後的優點

| 項目 | 說明 |
| --- | --- |
| 行為切換 | 於 Main Method 中直接透過注入不同攻擊行為物件，輕鬆切換行為 |
| 擴充容易 | 新增攻擊類別只需創建新類別並實作介面，不需修改 Hero 類別 |
| 維護性提升 | Hero 類別不再負責具體攻擊細節，程式碼更清晰易於維護 |
| 行為封裝 | 攻擊行為封裝於獨立類別，遵守單一職責原則 (Single Responsibility Principle) |

---

## 策略模式的典型組成元素

| 元素 | 說明 |
| --- | --- |
| Context | 代表使用策略的物件，擁有可變行為，透過持有 Strategy 介面引用委派行為 |

| | |
|---|---|
| Strategy 介面 | 定義演算法或行為的共同行為，例：Attack(Hero attacker, Hero target) |
| Concrete Strategy | 實作 Strategy 介面的具體行為類別，如 Waterball、Fireball、Earth、Powerball 等等 |

---

## 策略模式應用時機

> 當你希望能夠輕鬆地抽換某種行為，或想要隱藏行為細節以提升維護性時，策略模式是理想選擇。

---

## 策略模式設計流程總結

1. **封裝行為變種**：將變動的攻擊行為分拆成獨立類別封裝
2. **抽象行為共性**：定義攻擊行為介面統一對外操作
3. **委派職責**：由 Hero 類別委派攻擊行為給介面實作物件
4. **依賴注入**：透過建構子或 Setter 注入具體攻擊行為類別物件，促進行為替換與擴展

---

## 重要概念重點

| 概念 | 解釋 |
|---|---|
| **行為變動性** (Behavioral Variation) | 程式中某行為會依條件或需求變化，須封裝以降低變動影響範圍 |
| **依賴反轉** (Dependency Inversion) | 高層模組不應依賴低層細節，兩者均依賴抽象，抽象不應依賴細節 |
| **依賴注入** (Dependency Injection) | 將物件依賴透過外部傳入，降低類別間耦合度 |
| **委派** (Delegation) | 一物件將某職責交由另一物件負責實作，讓職責分工清晰且靈活 |

---

## 策略模式在本案例的關鍵程式碼片段示意（非完整碼）

```
// 攻擊行為介面 (Strategy)
public interface AttackType {
    void attack(Hero attacker, Hero target);
}
```

Thetawave.ai

```
// 具體攻擊策略 (Concrete Strategy)
public class Waterball implements AttackType {
    public void attack(Hero attacker, Hero target) {
        int damage = (int)(attacker.getHP() * 0.5);
        target.decreaseHP(damage);
    }
}

// Hero 類別 (Context)
public class Hero {
    private AttackType attackType;
    private int HP = 500;

    public Hero(AttackType attackType) {
        this.attackType = attackType;
    }

    public void attack(Hero target) {
        attackType.attack(this, target);
    }

    // HP getter, setter 等 ...
}
```

---

### 策略模式結果與實踐意義

策略模式將原本直接寫死於 Hero 類別的多變攻擊行為，拆分封裝至獨立的攻擊策略類別中，並透過抽象介面統一管理行為，Hero 類別只需持有該介面引用並委派行為。如此，英雄的攻擊行為變得易於擴充與切換，英雄類別本身程式碼保持簡潔，維護性與擴充性大幅提升。客戶端只要透過依賴注入傳入不同攻擊行為物件，即可自由組合不同的戰鬥策略，展現靈活多變的遊戲玩法。策略模式的使用使得程式架構更具彈性，行為變動被有效封裝，符合物件導向設計的核心原則。

## 2.7 補帖 - 無招勝有招_依賴反轉之重構三步驟.mp4

## 🎯 依賴反轉之重構三步驟核心解讀

### 依賴反轉 (Dependency Inversion) 設計原則

> **定義：**
> 核心（高層次）模組不能依賴衍生（低層次）模組，兩者應該依賴相同的抽象。抽象不應依賴細節，細節應該依賴抽象。

- **核心模組（高層次）**：系統中重要且穩定的部分，如遊戲中的角色 (hero) 類別。
- **衍生模組（低層次）**：可變動、易擴充的部分，如角色的多種攻擊行為。

- **抽象（介面）**：介於核心與衍生模組間的橋樑，定義共同的行為規範。

Thetawave.ai

---

## 問題背景：核心模組依賴衍生模組的問題

| 依賴方向 | 影響說明 |
| --- | --- |
| 核心 → 衍生 | 擴充衍生行為必須修改核心模組，導致核心模組維護困難、程式碼臃腫且錯誤率高 |

- **例子：**
  遊戲中角色 hero 直接實作攻擊行為，新增攻擊技能必須修改 hero 類別，破壞維護性。

---

## 依賴反轉之重構三步驟（Reconstruction Three Steps）

| 步驟 | 內容說明 | 目的與效果 |
| --- | --- | --- |
| 1. 封裝變動處 | 為每個行為變種開一個類別，封裝具體行為。 | 把變動的行為抽離核心模組，避免核心模組直接依賴具體行為。 |
| 2. 萃取共同行為 | 從變種類別中抽取共同行為，定義抽象介面（放在核心模組）。 | 將依賴反轉，核心與衍生模組都依賴抽象介面，確保擴充性與穩定性。 |
| 3. 委派 | 核心模組改為依賴抽象介面，將行為委派給實作該介面的衍生模組。 | 核心模組不再依賴衍生模組，實現依賴反轉設計原則。 |

---

## 詳細解說三步驟

1. 封裝變動處（Encapsulate What Varies）

- **說明：**
  將多種不同的行為變種獨立封裝為不同類別，例如水球攻擊、火球攻擊、地球攻擊各自成為獨立類別。
- **操作簽章（簽名）設計：**
  為每個行為定義最小且恰當的操作簽章（方法參數），保持封裝隱私與精準度。
  - 水球攻擊：攻擊單一怪物 → 傳入 1 個怪物
  - 火球攻擊：攻擊 1~5 個怪物 → 傳入 1~5 個怪物
  - 地球攻擊：隨機攻擊多個怪物 → 傳入多個怪物
- **比喻：**
  就像包裹上的標籤，操作簽章是包裹的標籤，讓外界知道包裹內裝了什麼，封裝更完整。

2. 萃取共同行為（Abstract Common Behaviors）

Thetawave.ai

- **說明：**
  從上述多個類別的操作簽章中，找出共同交集，定義一個抽象介面（如 AttackType）。
  - 由於操作參數差異（1 個、1~5 個、多個怪物），取「最大需求」作交集，即使用可接受多個怪物的參數。
- **抽象介面擺放位置：**
  須放置於核心模組中，確保衍生模組依賴核心模組的介面。
- **效果：**
  抽象介面成為雙方依賴的共同基礎，實作此介面的衍生類別必須符合該抽象。

3. 委派（Delegate）

- **說明：**
  核心模組 (hero) 不再直接實作攻擊行為，而是持有該抽象介面的參考，呼叫介面方法完成攻擊。
  - hero 類別與抽象介面形成依賴或關聯關係。
  - 實際行為由衍生模組中具體類別實作。
- **結果：**
  核心模組與衍生模組的依賴方向由「核心→衍生」反轉成「衍生→核心抽象」，達成依賴反轉設計原則。

---

## 依賴反轉與設計模式的關係

| 設計模式 | 重點 | 依賴反轉三步驟的角色 |
| --- | --- | --- |
| 策略模式 | 行為變動與擴充 | 透過三步驟完成抽象介面，讓核心角色依賴介面，行為實作獨立。 |
| 責任鏈模式 | 多行為變種串聯組合 | 依賴反轉三步驟完成抽象分離，並額外建立遞迴關聯串起多個處理者。 |

- **觀察：**
  設計模式中解決變動性與擴充性的問題，核心方法皆是利用依賴反轉三步驟完成抽象分離與依賴反轉。

---

## 重要概念與術語整理

| 概念 | 解釋 |
| --- | --- |
| **Context（上下文）** | 進行重構或應用設計模式的範圍，如 hero 類別中攻擊行為的部分。 |
| **變種（Variant）** | 同類型行為中最小變化單位，如不同攻擊技能或角色。 |
| **行為變動性（Behavioral Variability）** | 行為可變動的特性，需封裝與抽象以因應擴充。 |

metawave.ai

| | |
|---|---|
| **擴充性（Extensibility）** | 可新增行為變種而不修改既有核心程式碼的能力。 |
| **操作簽章（Method Signature）** | 方法的名稱與參數列表，代表行為的接口規範。 |
| **抽象介面（Interface/抽象類別）** | 定義共同行為的契約，放置於核心模組，供核心與衍生模組依賴。 |
| **委派（Delegation）** | 核心模組將行為執行權交給抽象介面實作，由衍生模組負責具體行為。 |

---

## 操作簽章取交集的原則

- 取交集時要「聽最多需求的那一方」，確保新介面能涵蓋所有衍生類別的行為需求。
- 例如：
  水球攻擊只需 1 個怪物參數，火球攻擊 1~5 個怪物，地球攻擊多個怪物，介面簽章定為「多個怪物」可覆蓋所有需求。

---

## 依賴反轉之重構三步驟執行流程

| 流程步驟 | 執行內容 |
|---|---|
| 定位 Context | 確認待分離的系統範圍與變動點，如 hero 類別的攻擊行為。 |
| 封裝每個行為變種 | 為每個具體行為開新類別、定義操作簽章並實作行為。 |
| 萃取共同行為抽象介面 | 根據操作簽章交集設計抽象介面，介面放入核心模組。 |
| 介面實作 | 由封裝的變種類別實作抽象介面。 |
| 核心模組委派行為 | hero 類別持有抽象介面參考，呼叫介面方法完成行為。 |
| 依賴方向反轉 | 依賴關係由「核心→衍生」變成「衍生→核心抽象」，實現依賴反轉原則。 |

---

## 依賴反轉三步驟的應用提醒

| 建議事項 | 理由 |
|---|---|
| 精準定位 Context | 不可亂套模式，確保重構範圍明確，避免設計混亂。 |
| 操作簽章定義要小氣且精準 | 小而精的簽章有助於後續抽象精準，避免過度泛化或不必要複雜度。 |
| 初期採用保守抽象策略 | 先學會基礎抽象再逐步增加彈性，避免一開始就追求過度靈活導致設計失衡。 |
| 慢工出細活 | 設計重點在於細心與精準，而非追求快速完成，類比機車駕照直線七秒的平衡技巧。 |

---

Thetawave.ai

### 依賴反轉三步驟在策略與責任鏈模式中的對比

| 模式 | 抽象位置 | 變種類別位置 | 依賴方向 | 額外結構特點 |
| --- | --- | --- | --- | --- |
| 策略模式 | 核心模組（介面） | 衍生模組 | 衍生模組依賴核心抽象 | 無特殊連結 |
| 責任鏈模式 | 核心模組（介面） | 衍生模組 | 衍生模組依賴核心抽象 | 變種類別間有遞迴關聯形成鏈條 |

- 責任鏈模式允許多個行為變種串聯參與決策，需多拉遞迴關聯來組合責任鏈。

---

### 依賴反轉設計的意涵

依賴反轉將系統中不穩定且易變的部分獨立封裝，並抽象成介面讓核心系統依賴該介面，促使系統擴充時無需改動核心邏輯，保證核心模組的穩定性與可維護性。所有衍生模組皆依賴核心模組中的抽象介面，這種雙向依賴抽象的結構使得系統既靈活又具備良好架構，並且可以在不破壞核心的情況下自由擴展衍生行為。